



Integration Test: Recap + From scratch task

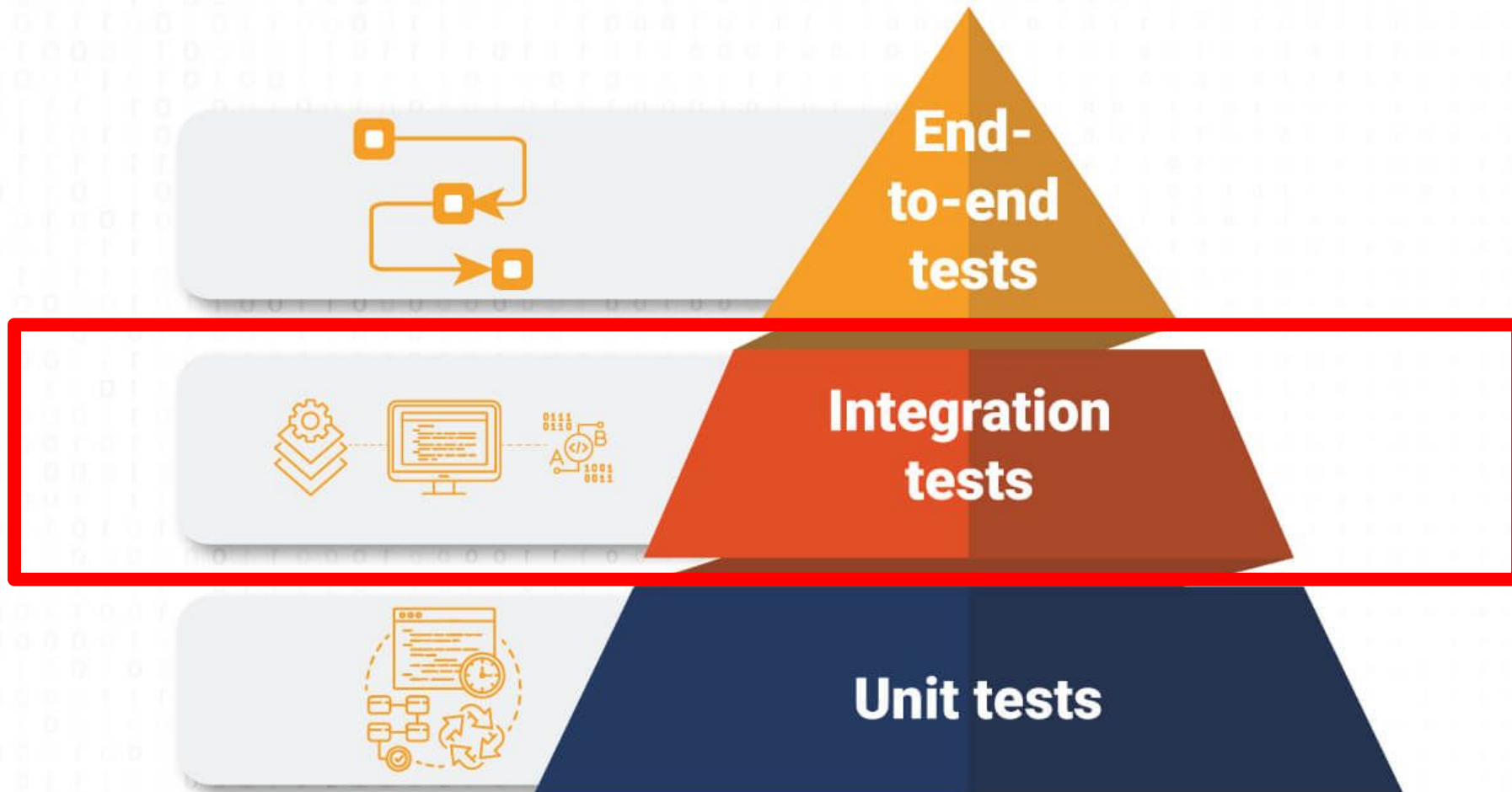


About me

- Name: Vitalii Vasylykha
- Software engineer

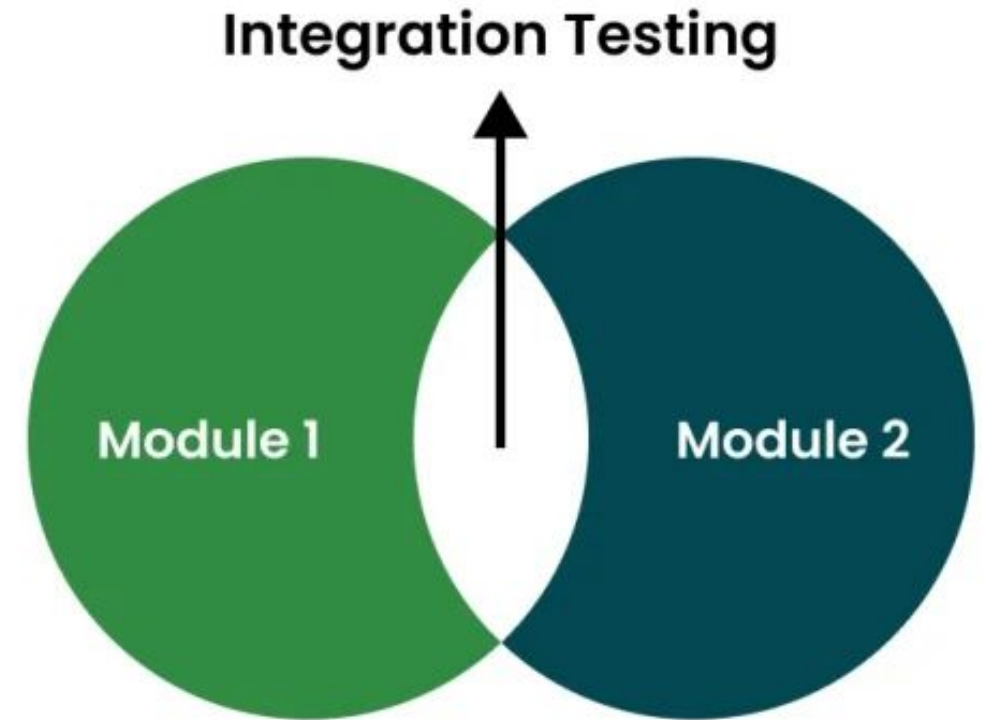


What is Integration testing?



What is Integration testing?

Integration testing refers to the process in which the interface of different applications, modules or components of a software application are tested as a combined group



What is Integration testing?

Aspect	Unit Testing	Integration Testing	End-to-End (E2E) Testing
Scope	Individual unit/component (e.g., function, class, module)	Multiple modules interacting	Entire application workflow from start to finish (user perspective)
Purpose	Verify individual components function correctly in isolation	Ensure different modules and interfaces work correctly together	Validate the complete user journey and overall system behavior
Environment	Development environment (often local)	Integration or QA test environment	Staging/Production-like environment
Dependencies	Uses mock data/objects to isolate the unit	Uses real test data; interacts with real, non-mocked systems (e.g., test database)	Uses data that is a copy of real production data; interacts with all real systems and network dependencies
Speed	Instant	Moderate	Long-running
Execution	Automated (typically run on every build)	Automated or manual	Primarily automated to replicate user scenarios
Bugs Caught	Catches bugs early at the root level	Catches interface and data flow issues between components	Catches issues that only appear when the entire system operates together
Scope	Individual unit/component (e.g., function, class, module)	Multiple units/modules interacting	Entire application workflow from start to finish (user perspective)
Purpose	Verify individual components function correctly in isolation	Ensure different modules and interfaces work correctly together	Validate the complete user journey and overall system behavior



Where Integration tests add value

Database integration (ORM, transactions)

Database migrations and schema validation

Event-driven systems

Application configuration and dependency injection

HTTP clients and external API

Payment processing flows



When Integration tests are not the right choice

Pure business logic

Simple input/output validation

UI interaction



Naming convention for Integration test classes

Suffix-based approach

Pattern	Example	Context & Usage
[...]IntegrationTest	UserRepositoryIntegrationTest	-
[...]IT	UserRepositoryIT	Default pattern for the Maven Failsafe Plugin
[...]IntTest	UserRepositoryIntTest	-



Naming convention for Integration test method

Should pattern

Pattern	Example	Context & Usage
should[Expected]When[Condition]	void shouldFailWhenProductIsOutOfStock():	Primarily used for Edge Cases, error scenarios, or specific business rules.
should[Expected]	void shouldCreateOrder():	Typically used for the Happy Path where the context is implied as "standard" or "valid".

Note: Always write the name so that it completes a logical sentence: "The system should [return 404] when [user ID is negative]."



Best Practice - Test Isolation & Independence

Tests should be able to run in any order, or even in parallel, without failing due to shared state.



Best Practice - Infrastructure Realism

If you use PostgreSQL in Production, use PostgreSQL in your integration tests.
Don't let an In-Memory DB hide your bugs.



Best Practice - Stable External Boundaries

Your tests should be able to run offline.
If an external API goes down, your CI/CD pipeline shouldn't.

**Mock Third-Party
APIs**



Best Practice - Execution Speed & Context Optimization

Fast tests are run often. Slow tests are skipped.
Optimize your context startup to keep developers productive.

Maximize
Context
Caching

Lean Test Slices



Resource

- Грокаємо тестування з Spring
- Integration Testing - Software Engineering
- Best Practices of Integration Testing



Thank you

- Author: Vitalii Vasylykha
- My LinkedIn: [vvasylykha](#)
- Date: March 2025
- [Join Codeus community in Discord](#)
- [Join Codeus community in LinkedIn](#)

